

Traccia Algoritmi - Esercizi 1A e 2A

Esercizio 1 (A) - Albero Binario

Si considerino le seguenti funzioni che implementano un Albero Binario in cui la parte informativa di ogni nodo è un numero intero:

- `createTree()` che restituisce un albero vuoto;
- `insert(A, x)` che inserisce il nodo `x` nell'albero `A`;
- `left(A)` che restituisce il sottoalbero sinistro di `A`;
- `right(A)` che restituisce il sottoalbero destro di `A`;
- `info(A)` che restituisce il valore associato al nodo radice di `A`;
- `setInfo(A, x)` che modifica il valore associato alla radice di `A` impostandolo a `x`;

`isNull(A)` restituisce `True` se l'albero `A` è vuoto e `False` altrimenti.

Dato un Albero Binario `A` con valori distinti, l'Antenato Comune Più Profondo (LCA) di due nodi con valore `x` e `y` è il nodo `v` di `A` tale che: (i) il nodo con valore `x` e il nodo con valore `y` appartengono entrambi al sottoalbero radicato in `v`, e (ii) nessun figlio di `v` soddisfa la proprietà (i). Un nodo è considerato antenato di se stesso.

1. Si realizzi una funzione `lca(A, x, y)` che, dato un Albero Binario `A` con valori distinti e due valori interi distinti `x` e `y` entrambi presenti in `A`, restituisca il valore `info(v)` del loro Antenato Comune Più Profondo. Per la realizzazione si può definire una funzione ausiliaria separata, da invocare all'interno di `lca`.
2. Si discuta, motivandola adeguatamente, la complessità temporale del metodo nei casi migliore e peggiore, esplicitando quale configurazione costituisca il caso migliore e il caso peggiore.
3. Si discuta, motivandola adeguatamente, la complessità spaziale del metodo nei casi migliore e peggiore.
4. Si discuta se, assumendo che `A` sia un Albero Binario di Ricerca, la complessità individuata cambierebbe, motivando la risposta e descrivendo come si modificherebbe l'algoritmo in tal caso.

Traccia Algoritmi - Esercizi 1A e 2A

Esercizio 2 (A) - Grafo non orientato e pesato

Siconsiderino le seguenti funzioni che implementano un grafo non orientato e pesato:

- `createGraph(m)`, che crea un grafo con m nodi identificati da interi in $[0, m - 1]$, senza archi;
- `insertEdge(G, x, y, w)`, che aggiunge un arco tra x e y di peso w (se esiste, ne aggiorna il peso);
- `deleteEdge(G, x, y)`, che rimuove, se presente, l'arco tra x e y ;
- `deleteNode(G, x)`, che rimuove il nodo x da G (solleva eccezione se ha archi incidenti o non esiste);
- `existsEdge(G, x, y)`, che restituisce `True` se esiste un arco tra x e y , `False` altrimenti;
- `getWeight(G, x, y)`, che restituisce il peso dell'arco (x, y) ;
- `size(G)`, che restituisce il numero di nodi del grafo;
- `nodes(G)`, che restituisce la lista degli identificatori dei nodi di G .

Si noti che, a differenza di altre tracce, l'API non fornisce una funzione `neighbors` per ottenere direttamente i vicini di un nodo: tale funzione, se necessaria, dovrà essere implementata a partire dalle primitive sopra elencate.

Un'azienda vuole assegnare i propri n dipendenti a due squadre (Squadra 0 e Squadra 1) in modo che, per ogni coppia di dipendenti in conflitto tra loro (rappresentata da un arco nel grafo G), i due dipendenti appartengano a squadre diverse. Si vuole determinare se tale assegnazione è possibile e, in caso affermativo, restituire la suddivisione ottenuta.

1. Si realizzi una funzione `neighbors(G, x)` che, dato un grafo G e un nodo x , restituisca la lista delle coppie $[y, w]$ tali che esiste l'arco (x, y) di peso w , utilizzando esclusivamente le primitive elencate sopra.
2. Si realizzi una funzione `coloraComponente(G, s, colori)` che, dato un grafo G , un nodo s e un vettore `colori` di dimensione `size(G)` inizializzato a `-1` (`-1` = non ancora assegnato), tenti di assegnare a ciascun nodo della componente connessa contenente s un colore in $\{0, 1\}$ usando una visita in ampiezza, in modo che ogni coppia di nodi adiacenti abbia colori diversi, aggiornando il vettore `colori`. La funzione deve restituire `True` se l'assegnazione è riuscita, `False` se la componente non è 2-colorabile (cioè contiene un ciclo di lunghezza dispari).
3. Si realizzi una funzione `isBipartite(G)` che, dato un grafo G , utilizzi la funzione `coloraComponente` del punto precedente per determinare se G è bipartito. In caso affermativo, la funzione deve restituire la coppia `(True, [squadra0, squadra1])`, dove `squadra0` e `squadra1` sono le liste dei nodi assegnati rispettivamente alla Squadra 0 e alla Squadra 1, oppure la coppia `(False, None)` altrimenti.
4. Si analizzi e si motivi la complessità temporale nel caso peggiore di `neighbors`, `coloraComponente` e `isBipartite`, specificando la struttura dati ausiliaria adottata.
5. Si analizzi e si motivi la complessità spaziale nel caso peggiore di `neighbors`, `coloraComponente` e `isBipartite`.